IC Design Lab
RISC-V Architecture, Group - 4
<u>Modules and Sub Modules</u>

```
`timescale 1ns / 1ps

// STAGE 1 : Fetch Stage
%%%
```

```
    pc u_pc0
    (
        .clk(clk),
        .rst(rst),
        .load(load),
        .next_sel(next_sel),
        .jalr(jalr),
        .dmem_valid(valid),
        .next_address(next_address),
        .branch_reselt(branch_reselt),
        .address_in(0),
        .address_out(address_out),
        .pre_address_pc(pre_address_pc)
    );

    always @ (*) begin
        if ((load && !valid)) begin
            mask = 4'b1111;
            we_re = 1'b0;
            request = 1'b0;
        end
        else begin
            mask = 4'b1111;
            we_re = 1'b0;
            request = 1'b1;
        end
    end
    always @ (*) begin
        instruction = instruction_fetch ;
    end
endmodule


// fetch_pipe.v
module fetch_pipe(
  input wire clk,
  input wire rst,
  input wire [31:0] pre_address_pc,
  input wire [31:0] instruction_fetch,
  input wire next_select,
```

```
  input wire branch_result,
  input wire jalr,
  input wire load,

  output wire [31:0] pre_address_out,
  output wire [31:0] instruction
);

  reg [31:0] pre_address, instruc;
  reg flush_pipeline , flush_pipeline2;

  always @ (posedge clk or negedge rst) begin
    if (!rst) begin
      pre_address     <= 32'b0;
      instruc         <= 32'b0;
      flush_pipeline  <= 0;
    end
    else begin
      if (next_select | branch_result | jalr) begin
      // If jal, jalr, or branch result is high, flush the pipeline for one cycle
      pre_address     <= 32'b0;
      instruc         <= 32'b0;
      flush_pipeline  <= 1; // Set flag to flush for one cycle
    end
    else begin
      if (flush_pipeline) begin
        // Stall the pipeline for one additional cycle after flushing
        pre_address     <= 32'b0;
        instruc         <= 32'b0;
        flush_pipeline  <= 0; // Reset flag after one cycle stall
        flush_pipeline2 <= 1; // Set flag to flush for one cycle
      end
      else if (flush_pipeline2) begin
        // Stall the pipeline for one additional cycle after flushing
        pre_address     <= 32'b0;
        instruc         <= 32'b0;
        flush_pipeline2 <= 0; // Reset flag after one cycle stall
      end
      else if (load) begin
        //stall pipeline
```

```
        pre_address     <= pre_address_out;
        instruc         <= instruction;
      end
      else begin
        // For other instructions, proceed normally
        pre_address     <= pre_address_pc;
        instruc         <= instruction_fetch;
      end
    end
    end
  end

  assign pre_address_out = pre_address;
  assign instruction     = instruc;
endmodule


// program_counter
module pc (
    input wire clk,
    input wire rst,
    input wire load,
    input wire jalr,
    input wire next_sel,
    input wire dmem_valid,
    input wire branch_reselt,
    input wire [31:0]next_address,
    input wire [31:0]address_in,

    output reg [31:0]address_out,
    output wire [31:0] pre_address_pc
);

    reg [31:0] pre_address;
    always @(posedge clk or negedge rst) begin
        if(!rst)begin
            address_out <= 0;
            pre_address  <= address_out;
        end
```

```
        else begin
            pre_address  <= address_out;
            if (next_sel | branch_reselt)begin
                address_out <= next_address;
            end
            else if (jalr)begin
                address_out <= next_address;
            end
            else if ((load && !dmem_valid))begin
                address_out <= address_out;
                pre_address <= pre_address_pc;
            end
            else begin
                address_out <= address_out + 32'd4;
            end
        end

    end

    assign pre_address_pc = pre_address;
endmodule


// instruc_memory_top
module instruc_mem_top #(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire rst,
    input wire we_re,
    input wire request,
    input wire [3:0]  mask,
    input wire [7:0]  address,
    input wire [31:0] data_in,

    output reg valid,
    output wire [31:0] data_out
    );

    always @(posedge clk or negedge rst ) begin
```

```
        if(!rst)begin
            valid <= 0;
        end
        else begin
            valid <= request;
        end
    end

    memory #(
      .INIT_MEM(INIT_MEM)
    )u_memory(
        .clk(clk),
        .we_re(we_re),
        .request(request),
        .mask(mask),
        .address(address),
        .data_in(data_in),
        .data_out(data_out)
    );
endmodule
```

```
// STAGE 2 : Decode Stage
%%%
```

```
input wire valid,
input wire reg_write_en_in,
input wire load_control_signal,
input wire [31:0] instruction,
input wire [31:0] pc_address,
input wire [31:0] rd_wb_data,
input wire [31:0] instruction_rd,

output wire load,
output wire store,
output wire jalr,
output wire next_sel,
output wire branch_result,
output wire reg_write_en_out,
output wire [4:0]  alu_control,
output wire [1:0]  mem_to_reg,
output wire [4:0]  rs1 , rs2,
output wire [31:0] opb_data,
output wire [31:0] opa_mux_out,
output wire [31:0] opb_mux_out
);

wire branch;
wire operand_a;
wire operand_b;
wire [2:0]  imm_sel;
wire [31:0] op_a , op_b;
wire [31:0] imm_mux_out;
wire [31:0] i_immo , s_immo , sb_immo , uj_immo , u_immo;

// CONTROL UNIT
controlunit u_cu0
(
    .opcode(instruction[6:0]),
    .fun3(instruction[14:12]),
    .fun7(instruction[31:25]),
    .valid(valid),
    .reg_write(reg_write_en_out),
    .imm_sel(imm_sel),
    .next_sel(next_sel),
```

```
    .operand_b(operand_b),
    .operand_a(operand_a),
    .mem_to_reg(mem_to_reg),
    .Load(load),
    .Store(store),
    .jalr_out(jalr),
    .Branch(branch),
    .load_control(load_control_signal),
    .alu_control(alu_control)
);

// IMMEDIATE GENERATION
immediategen u_imm_gen0 (
    .instr(instruction),
    .i_imme(i_immo),
    .sb_imme(sb_immo),
    .s_imme(s_immo),
    .uj_imme(uj_immo),
    .u_imme(u_immo)
);

//IMMEDIATE SELECTION MUX
mux3_8 u_mux0(
    .a(i_immo),
    .b(s_immo),
    .c(sb_immo),
    .d(uj_immo),
    .e(u_immo),
    .sel(imm_sel),
    .out(imm_mux_out)
);

// REGISTER FILE
registerfile u_regfile0
(
    .clk(clk),
    .rst(rst),
    .en(reg_write_en_in),
    .rs1(instruction[19:15]),
    .rs2(instruction[24:20]),
```

```
        .rd(instruction_rd[11:7]),
        .data(rd_wb_data),
        .op_a(op_a),
        .op_b(op_b)
    );

    assign rs1 = instruction[19:15];
    assign rs2 = instruction[24:20];
    assign opb_data = op_b ;

    //SELECTION OF PROGRAM COUNTER OR OPERAND A
    mux u_mux1
    (
        .a(op_a),
        .b(pc_address),
        .sel(operand_a),
        .out(opa_mux_out)
    );

    //SELECTION OF OPERAND B OR IMMEDIATE
    mux u_mux2(
        .a(op_b),
        .b(imm_mux_out),
        .sel(operand_b),
        .out(opb_mux_out)
    );

    //BRANCH
    branch u_branch0(
        .en(branch),
        .op_a(op_a),
        .op_b(op_b),
        .fun3(instruction[14:12]),
        .result(branch_result)
    );
endmodule


// decode_pipe
module  decode_pipe(
```

```
 input wire clk,
 input wire rst,
 input wire load_in,
 input wire store_in,
 input wire jalr_in,
 input wire next_sel_in,
 input wire branch_result_in,
 input wire reg_write_in,
 input wire [4:0] rs1_in,
 input wire [4:0] rs2_in,
 input wire [4:0] alu_control_in,
 input wire [1:0]  mem_to_reg_in,
 input wire [31:0] opa_mux_in,
 input wire [31:0] opb_mux_in,
 input wire [31:0] opb_data_in,
 input wire [31:0] pre_address_in,
 input wire [31:0] instruction_in,

 output wire load,
 output wire store,
 output wire jalr_out,
 output wire next_sel,
 output wire branch_result,
 output wire reg_write_out,
 output wire [4:0] rs1_out,
 output wire [4:0] rs2_out,
 output wire [4:0] alu_control,
 output wire [1:0]  mem_to_reg,
 output wire [31:0] opa_mux_out,
 output wire [31:0] opb_mux_out,
 output wire [31:0] opb_data_out,
 output wire [31:0] pre_address_out,
 output wire [31:0] instruction_out
);

 reg l,s,nextsel,branch_res,jalr;
 reg reg_write;
 reg [1:0] mem_reg;
 reg [4:0] alu_con;
 reg [4:0] rs1;
```

```
reg [4:0] rs2;
reg [31:0] opa_mux,opb_mux,opb_data,pre_address,instruction;

always @ (posedge clk or negedge rst) begin
  if (!rst) begin
    l          <= 0;
    s          <= 0;
    jalr       <= 0;
    nextsel    <= 0;
    branch_res <= 0;
    mem_reg    <= 0;
    alu_con    <= 0;
    opa_mux    <= 0;
    opb_mux    <= 0;
    opb_data   <= 0;
    pre_address <= 0;
    instruction <= 0;
    reg_write  <= 0;
    rs1        <= 0;
    rs2        <= 0;
  end
  else begin
    l          <= load_in;
    s          <= store_in;
    jalr       <= jalr_in;
    nextsel    <= next_sel_in;
    branch_res <= branch_result_in;
    mem_reg    <= mem_to_reg_in;
    alu_con    <= alu_control_in;
    opa_mux    <= opa_mux_in;
    opb_mux    <= opb_mux_in;
    opb_data   <= opb_data_in;
    pre_address <= pre_address_in;
    instruction <= instruction_in;
    reg_write  <= reg_write_in;
    rs1        <= rs1_in;
    rs2        <= rs2_in;
  end
end
```

```
  assign load            = l;
  assign store           = s;
  assign rs1_out         = rs1;
  assign rs2_out         = rs2;
  assign jalr_out        = jalr;
  assign next_sel        = nextsel;
  assign reg_write_out   = reg_write;
  assign branch_result   = branch_res;
  assign mem_to_reg      = mem_reg;
  assign alu_control     = alu_con;
  assign opa_mux_out     = opa_mux;
  assign opb_mux_out     = opb_mux;
  assign opb_data_out    = opb_data;
  assign instruction_out = instruction;
  assign pre_address_out = pre_address;
endmodule


// immediate_gen
module immediategen (
   input wire [31:0]instr,
   output reg [31:0] i_imme,
   output reg [31:0] s_imme,
   output reg [31:0] sb_imme,
   output reg [31:0] uj_imme,
   output reg [31:0] u_imme
);

   always @(*) begin
      i_imme  = {{20{instr[31]}}, instr[31:20]};
      s_imme  = {{20{instr[31]}}, instr[31:25], instr[11:7]};
      sb_imme = {{19{instr[31]}},instr[31],instr[7],instr[30:25],instr[11:8],1'b0};
      uj_imme = {{11{instr[31]}},instr[31],instr[19:12],instr[20],instr[30:21],1'b0};
      u_imme  = {{instr[31:12]},12'b0};
   end
endmodule
```

```
// STAGE 3 : Execute stage
%%%
```

```
            register[rd] <= data;
        end
    end
end

assign op_a = (rs1 != 0)? register[rs1] : 0;
assign op_b = (rs2 != 0)? register[rs2] : 0;
endmodule


// execute
module execute (
    input wire [31:0] a_i,
    input wire [31:0] b_i,
    input wire [4:0]  alu_control,
    input wire [31:0] pc_address,

    output wire [31:0] alu_res_out,
    output wire [31:0] next_sel_address
    );

    // ALU
    alu u_alu0
    (
        .a_i(a_i),
        .b_i(b_i),
        .op_i(alu_control),
        .res_o(alu_res_out)
    );
    //adder
    adder u_adder0(
        .a(pc_address),
        .adder_out(next_sel_address)
    );
endmodule


// execute_pipe
module execute_pipe(
  input wire clk,
```

```
input wire rst,
input wire load_in,
input wire store_in,
input wire reg_write_in,
input wire [31:0] opb_datain,
input wire [31:0] alu_res,
input wire [1:0] mem_reg_in,
input wire [31:0] next_sel_addr,
input wire [31:0] pre_address_in,
input wire [31:0] instruction_in,

output wire reg_write_out,
output wire load_out,
output wire store_out,
output wire [31:0] opb_dataout,
output wire [31:0] alu_res_out,
output wire [1:0] mem_reg_out,
output wire [31:0] next_sel_address,
output wire [31:0] pre_address_out,
output wire [31:0] instruction_out
);

reg load , store , reg_write;
reg [1:0] mem_reg;
reg [31:0] alu_result , nextsel_addr;
reg [31:0] pre_address , instruction , opb_data;

always @ (posedge clk or negedge rst) begin
  if (!rst) begin
    load          <= 0;
    store         <= 0;
    mem_reg       <= 0;
    opb_data      <= 0;
    pre_address   <= 0;
    instruction   <= 0;
    alu_result    <= 0;
    nextsel_addr  <= 0;
    reg_write     <= 0;
  end
  else begin
```

```
      load          <= load_in;
      store         <= store_in;
      mem_reg       <= mem_reg_in;
      opb_data      <= opb_datain;
      pre_address   <= pre_address_in;
      instruction   <= instruction_in;
      alu_result    <= alu_res;
      nextsel_addr  <= next_sel_addr;
      reg_write     <= reg_write_in;
    end
  end

  assign reg_write_out    = reg_write;
  assign load_out         = load;
  assign store_out        = store;
  assign mem_reg_out      = mem_reg;
  assign opb_dataout      = opb_data;
  assign alu_res_out      = alu_result;
  assign pre_address_out  = pre_address;
  assign instruction_out  = instruction;
  assign next_sel_address = nextsel_addr;
endmodule


// alu.v
module alu (a_i,b_i,op_i,res_o);

    input wire [31:0]a_i;
    input wire [31:0]b_i;
    input wire [4:0]op_i;

    output reg [31:0]res_o;

    reg [63:0] res_64;

    always @(*) begin
        if (op_i==5'b00000) begin
            res_o = a_i + b_i; //add
        end
        else if (op_i==5'b00001) begin
```

```
    res_o = a_i - b_i; //sub
end
else if (op_i==5'b00010) begin
    res_o = a_i << b_i; //shift left logical
end
else if (op_i==5'b00011) begin
    res_o = $signed (a_i) < $signed (b_i); //shift less then
end
else if (op_i==5'b00100) begin
    res_o = a_i < b_i; //shift less then unsigned
end
else if (op_i==5'b00101) begin
    res_o = a_i ^ b_i; //xor
end
else if (op_i==5'b00110) begin
    res_o = a_i >> b_i; //shift right logical
end
else if (op_i==5'b00111) begin
    res_o = a_i >>> b_i; //shift right arithematic
end
else if (op_i==5'b01000) begin
    res_o = a_i | b_i; //or
end
else if (op_i==5'b01001) begin
    res_o = a_i & b_i; //and
end
else if (op_i==5'b01111) begin
    res_o = b_i; //for lui
end
//----------M Extension------------//
else if (op_i==5'b10000) begin
    res_64= (a_i * b_i);//for MUL
    res_o = res_64[31:0]; //lower bits
end
else if (op_i==5'b10001) begin
    res_64= ($signed (a_i) * $signed (b_i));//for MULH
    res_o = res_64[63:32]; //upper bits
end
else if (op_i==5'b10010) begin
    res_64= ($signed (a_i) * (b_i));//for MULHSU
```

```
            res_o = res_64[63:32]; //upper bits
        end
        else if (op_i==5'b10011) begin
            res_64= a_i * b_i;     //for MULU
            res_o = res_64[63:32]; //upper bits
        end
        else if (op_i==5'b10100) begin
            res_o = ($signed (a_i) / $signed (b_i)); //for DIV
        end
        else if (op_i==5'b10101) begin
            res_o = a_i / b_i; //for DIVU
        end
        else if (op_i==5'b10110) begin
            res_o = ($signed (a_i) % $signed (b_i)); //for REM
        end
        else if (op_i==5'b10111) begin
            res_o = a_i % b_i; //for REMU
        end
        else begin
            res_o = 0;
        end
    end
endmodule


// adder.v
module adder (a,adder_out);

    input wire [31:0] a;
    output reg [31:0] adder_out;

    always @ (*) begin
        adder_out = a + 32'd4;
    end
endmodule


// branch.v
module branch (op_a,op_b,fun3,en,result);
```

```
  input [31:0]op_a,op_b;
  input [2:0]fun3;
  input en;

  output reg result;

  always @(*) begin
    if(en==1)begin
      case (fun3)
        3'b000 : result = (op_a == op_b) ? 1 : 0 ;
        3'b001 : result = (op_a != op_b) ? 1 : 0 ;
        3'b100 : result = ($signed (op_a) < $signed (op_b)) ? 1 : 0 ;
        3'b101 : result = ($signed (op_a) >= $signed (op_b)) ? 1 : 0 ;
        3'b110 : result = (op_a < op_b) ? 1 : 0 ;
        3'b111 : result = (op_a >= op_b) ? 1 : 0 ;
      endcase
    end
    else begin
      result = 0;
    end
  end

endmodule
```

```
// STAGE 4 : Control
%%%
```

```
// control_unit
module controlunit (
    input wire [6:0] opcode,
    input wire [2:0] fun3,
    input wire [6:0]fun7,
    input wire valid,
    input wire load_control,

    output wire reg_write,
    output wire [2:0]imm_sel,
    output wire operand_b,
    output wire operand_a,
    output wire [1:0] mem_to_reg,
    output wire Load,
    output wire jalr_out,
    output wire Store,
    output wire Branch,
    output wire mem_en,
    output wire next_sel,
    output wire [4:0] alu_control
);

    wire r_type;
    wire i_type;
    wire load;
    wire store;
    wire branch;
    wire jal;
    wire jalr;
    wire lui;
    wire auipc;

    type_decoder u_typedec0 (
        .opcode(opcode),
        .valid(valid),
        .r_type(r_type),
        .i_type(i_type),
        .load(load),
        .branch(branch),
```

```
        .store(store),
        .jal(jal),
        .jalr(jalr),
        .lui(lui),
        .auipc(auipc),
        .load_signal_controller(load_control)
    );

    control_decoder u_controldec0 (
        .fun3(fun3),
        .fun7(fun7),
        .i_type(i_type),
        .r_type(r_type),
        .load(load),
        .store(store),
        .branch(branch),
        .jal(jal),
        .jalr(jalr),
        .lui(lui),
        .auipc(auipc),
        .next_sel(next_sel),
        .Branch(Branch),
        .Load(Load),
        .Store(Store),
        .jalr_out(jalr_out),
        .mem_to_reg(mem_to_reg),
        .reg_write(reg_write),
        .mem_en(mem_en),
        .operand_b(operand_b),
        .operand_a(operand_a),
        .imm_sel(imm_sel),
        .alu_control(alu_control),
        .load_control(load_control)
    );

endmodule


// control_decoder
module control_decoder (
```

```
    input wire [2:0] fun3,
    input wire [6:0]fun7,
    input wire i_type,
    input wire r_type,
    input wire load,
    input wire store,
    input wire branch,
    input wire jal,
    input wire jalr,
    input wire lui,
    input wire auipc,
    input wire load_control,

    output reg Load,
    output reg Store,
    output reg jalr_out,
    output reg [1:0] mem_to_reg,
    output reg reg_write,
    output reg mem_en,
    output reg operand_b,
    output reg operand_a,
    output reg [2:0]imm_sel,
    output reg Branch,
    output reg next_sel,
    output reg [4:0]alu_control
);
wire fun_7 = fun7[5];

always @(*) begin
    //reg write signal for register file
    reg_write = r_type | i_type | load | jal | jalr | lui | auipc | load_control;
    //operand a select for first input of alu
    operand_a = branch | jal | auipc;
    //operand b signal for second input of alu
    operand_b = i_type | load | store | branch | jal | jalr | lui | auipc;
    //load
    Load = load;
    //store
    Store = store;
    //branch
```

```
Branch =  branch;
//selection for next address if any jump instrucion run
next_sel = jal;
jalr_out = jalr;
//mem enable
mem_en = store;

if(r_type)begin //rtype
    mem_to_reg = 2'b00;
    if(fun3==3'b000 & fun7==0000000)begin
        alu_control = 5'b00000;
    end
    else if(fun3==3'b000 & fun7==7'b0100000)begin
        alu_control = 5'b00001;
    end
    else if (fun3==3'b001 & fun7==7'b0000000)begin
        alu_control = 5'b00010;
    end
    else if (fun3==3'b010 & fun7==7'b0000000)begin
        alu_control = 5'b00011;
    end
    else if (fun3==3'b011 & fun7==7'b0000000)begin
        alu_control = 5'b00100;
    end
    else if (fun3==3'b100 & fun7==7'b0000000)begin
        alu_control = 5'b00101;
    end
    else if (fun3==3'b101 & fun7==7'b0000000)begin
        alu_control = 5'b00110;
    end
    else if (fun3==3'b101 & fun7==7'b0100000)begin
        alu_control = 5'b00111;
    end
    else if (fun3==3'b110 & fun7==7'b0000000)begin
        alu_control = 5'b01000;
    end
    else if (fun3==3'b111 & fun7==7'b0000000)begin
        alu_control = 5'b01001;
    end
    //--------M-Extension------------//
```

```
    else if (fun3==3'b000 & fun7==7'b0000001)begin
        alu_control = 5'b10000;
    end
    else if (fun3==3'b001 & fun7==7'b0000001)begin
        alu_control = 5'b10001;
    end
    else if (fun3==3'b010 & fun7==7'b0000001)begin
        alu_control = 5'b10010;
    end
    else if (fun3==3'b011 & fun7==7'b0000001)begin
        alu_control = 5'b10011;
    end
    else if (fun3==3'b100 & fun7==0000001)begin
        alu_control = 5'b10100;
    end
    else if (fun3==3'b101 & fun7==7'b0000001)begin
        alu_control = 5'b10101;
    end
    else if (fun3==3'b110 & fun7==7'b0000001)begin
        alu_control = 5'b10110;
    end
    else if (fun3==3'b111 & fun7==7'b0000001)begin
        alu_control = 5'b10111;
    end
end
else if (i_type)begin //itype
    imm_sel = 3'b000; //i_type selection
    mem_to_reg = 2'b00;
    if(fun3==3'b000)begin
        alu_control = 5'b00000;
    end
    else if (fun3==3'b001 & fun_7==0)begin
        alu_control = 5'b00010;
    end
    else if (fun3==3'b010 & fun_7==0)begin
        alu_control = 5'b00011;
    end
    else if (fun3==3'b011 & fun_7==0)begin
        alu_control = 5'b00100;
    end
```

```
    else if (fun3==3'b100 & fun_7==0)begin
        alu_control = 5'b00101;
    end
    else if (fun3==3'b101 & fun_7==0)begin
        alu_control = 5'b00110;
    end
    else if (fun3==3'b101 & fun_7==1)begin
        alu_control = 5'b00111;
    end
    else if (fun3==3'b110 & fun_7==0)begin
        alu_control = 5'b01000;
    end
    else if (fun3==3'b111 & fun_7==0)begin
        alu_control = 5'b01001;
    end
end
else if (store) begin //store
    imm_sel = 3'b001; //store selection
    mem_to_reg = 2'b00;
    if (fun3==3'b000)begin //sb
        alu_control = 5'b00000;
        //signal = 2'b00;
    end
    else if (fun3==3'b001)begin //sh
        alu_control = 5'b00000;
        //signal = 2'b01;
    end
    else if (fun3==3'b010)begin //sw
        alu_control = 5'b00000;
        //signal = 2'b10;
    end
end
else if (load) begin
    imm_sel = 3'b000; //i_type selection
    mem_to_reg = 2'b01;
    if (fun3==3'b000)begin //lb
        alu_control = 5'b00000;
    end
    else if(fun3==3'b001)begin //lh
        alu_control = 5'b00000;
```

```
        end
        else if(fun3==3'b010)begin //lw
            alu_control = 5'b00000;
        end
        else if(fun3==3'b100)begin //lbu
            alu_control = 5'b00000;
        end
        else if(fun3==3'b101)begin //lhu
            alu_control = 5'b00000;
        end
        else if(fun3==3'b110)begin //lwu
            alu_control = 5'b00000;
        end
    end
    else if (branch)begin
        alu_control = 5'b00000;
        mem_to_reg = 2'b00;
        imm_sel = 3'b010; //branch selection
    end
    else if (jal)begin
        alu_control = 5'b00000;
        mem_to_reg = 2'b10;
        imm_sel = 3'b011; //jal selection
    end
    if(jalr)begin
        mem_to_reg = 2'b00;
        alu_control = 5'b00000;
        imm_sel = 3'b000;//i_type selection
    end
    else if(lui)begin
        mem_to_reg = 2'b00;
        imm_sel = 3'b100;//u_type selection
        alu_control = 5'b01111;
    end
    else if(auipc)begin
        mem_to_reg = 2'b00;
        alu_control = 5'b00000;
        imm_sel = 3'b100;//u_type selection
    end
end
```

```
endmodule


// type_decoder
module type_decoder
(opcode,r_type,i_type,load,store,branch,jal,jalr,lui,auipc,valid,load_signal_controller);

    input wire [6:0]opcode;
    input wire valid;
    input wire load_signal_controller;

    output reg r_type;
    output reg i_type;
    output reg load;
    output reg store;
    output reg branch;
    output reg jal;
    output reg jalr;
    output reg lui;
    output reg auipc;

    always @(*)begin
        r_type = 1'b0;
        i_type = 1'b0;
        store = 1'b0;
        load = 1'b0;
        branch = 1'b0;
        auipc = 1'b0;
        jal = 1'b0;
        jalr = 1'b0;
        lui = 1'b0;
        case(opcode)
            7'b0110011:begin
                r_type = 1'b1;
            end
            7'b0010011:begin
                i_type = 1'b1;
            end
            7'b0100011:begin
```

```
            store = 1'b1;
        end
        7'b0000011:begin
            if (valid | load_signal_controller) begin
                load = 1'b0;
            end
            else begin
                load = 1'b1;
            end
        end
        7'b1100011:begin
            branch = 1'b1;
        end
        7'b0010111:begin
            auipc = 1'b1;
        end
        7'b1101111:begin
            jal = 1'b1;
        end
        7'b1100111:begin
            jalr = 1'b1;
        end
        7'b0110111:begin
            lui = 1'b1;
        end

        default:begin
            r_type = 1'b0;
            i_type = 1'b0;
            store= 1'b0;
            load = 1'b0;
            branch = 1'b0;
            auipc = 1'b0;
            jal = 1'b0;
            jalr = 1'b0;
            lui = 1'b0;
        end
    endcase
  end
endmodule
```

```
// STAGE 5 : Memory
%%%
```

```
    wrappermem u_wrap_mem0 (
        .data_i(op_b),
        .byteadd(alu_out_address [1:0]),
        .fun3(instruction [14:12]),
        .mem_en(store),
        .Load(load),
        .wrap_load_in(wrap_load_in),
        .masking(mask),
        .data_valid(data_valid),
        .data_o(store_data_out),
        .wrap_load_out(wrap_load_out)
    );

    always @ (*) begin
        if (!valid & !load & !store) begin
            request = 0;
            we_re = 0;
        end
        else begin
            request = load | store ;
            we_re = store ;
        end
    end
endmodule


// memory
module memory#(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire we_re,
    input wire request,
    input wire [7:0]address,
    input wire [31:0]data_in,
    input wire [3:0]mask,

    output reg [31:0]data_out
);
```

```
    reg [31:0] mem [0:255];

    initial begin
        if (INIT_MEM)
            $readmemh("tb/instr.mem",mem);
    end

    always @(posedge clk) begin
        if (request && we_re) begin
            if(mask[0]) begin
                mem[address][7:0] <= data_in[7:0];
            end
            if(mask[1]) begin
                mem[address][15:8] <= data_in[15:8];
            end
            if(mask[2]) begin
                mem[address][23:16] <= data_in[23:16];
            end
            if(mask[3]) begin
                mem[address][31:24] <= data_in[31:24];
            end
        end

        else begin
            if (request && !we_re) begin
                data_out <= mem[address];
            end
        end
    end
endmodule


// memstage_pipe

module memory_pipe(
  input wire clk,
  input wire rst,
  input wire reg_write_in,
  input wire [1:0]  mem_reg_in,
  input wire [31:0] wrap_load_in,
```

```
input wire [31:0] alu_res,
input wire [31:0] next_sel_addr,
input wire [31:0] instruction_in,
input wire [31:0] pre_address_in,

output wire reg_write_out,
output wire [31:0] alu_res_out,
output wire [1:0]  mem_reg_out,
output wire [31:0] next_sel_address,
output wire [31:0] wrap_load_out,
output wire [31:0] instruction_out,
output wire [31:0] pre_address_out
);

reg reg_write;
reg [1:0]  mem_reg;
reg [31:0] pre_address_pc;
reg [31:0] alu_result , nextsel_addr , wrap_load , instruction;

always @ (posedge clk) begin
  if (!rst) begin
    mem_reg        <= 0;
    alu_result     <= 0;
    nextsel_addr   <= 0;
    wrap_load      <= 0;
    instruction    <= 0;
    reg_write      <= 0;
    pre_address_pc <= 0;
  end
  else begin
    mem_reg        <= mem_reg_in;
    alu_result     <= alu_res;
    nextsel_addr   <= next_sel_addr;
    wrap_load      <= wrap_load_in;
    instruction    <= instruction_in;
    reg_write      <= reg_write_in;
    pre_address_pc <= pre_address_in;
  end
end
```

```
  assign reg_write_out    = reg_write;
  assign mem_reg_out      = mem_reg;
  assign alu_res_out      = alu_result;
  assign next_sel_address = nextsel_addr;
  assign wrap_load_out    = wrap_load;
  assign instruction_out  = instruction;
  assign pre_address_out  = pre_address_pc;
endmodule


// data_memory_top
module data_mem_top #(
    parameter INIT_MEM = 0
)(
    input wire clk,
    input wire rst,
    input wire we_re,
    input wire request,
    input wire load,
    input wire [3:0]  mask,
    input wire [7:0]  address,
    input wire [31:0] data_in,

    output reg valid,
    output wire [31:0] data_out
    );

    always @(posedge clk or negedge rst ) begin
        if(!rst)begin
            valid <= 0;
        end
        else begin
            valid <= load;
        end
    end

    memory #(
      .INIT_MEM(INIT_MEM)
    )u_memory(
        .clk(clk),
```

```
        .we_re(we_re),
        .request(request),
        .mask(mask),
        .address(address),
        .data_in(data_in),
        .data_out(data_out)
    );
endmodule


// wrapper_memory
module wrappermem (
    input wire [31:0] data_i,
    input wire [1:0] byteadd,
    input wire [2:0] fun3,
    input wire mem_en,
    input wire Load,
    input wire data_valid,
    input wire [31:0]wrap_load_in,

    output reg [3:0] masking,
    output reg [31:0] data_o,
    output reg [31:0] wrap_load_out

);

    always @(*) begin
        if (mem_en) begin
            masking = 4'b0000;
            if(fun3==3'b000)begin //sb
              case (byteadd)
                  00: begin
                      masking = 4'b0001;
                      data_o = data_i;
                  end
                  01: begin
                      masking = 4'b0010;
                      data_o = {data_i[31:16],data_i[7:0],data_i[7:0]};
                  end
                  10: begin
```

```
            masking = 4'b0100;
            data_o = {data_i[31:24],data_i[7:0],data_i[15:0]};
         end
         11: begin
            masking = 4'b1000;
            data_o = {data_i[7:0],data_i[23:0]};
         end
      endcase
   end

   if(fun3==3'b001)begin //sh
      case (byteadd)
         00: begin
            masking = 4'b0011;
            data_o = data_i;
         end
         01: begin
            masking = 4'b0110;
            data_o = {data_i[31:24],data_i[15:0],data_i[7:0]};
         end
         10: begin
            masking = 4'b1100;
            data_o = {data_i[15:0],data_i[15:0]};
         end
      endcase
   end

   if(fun3==3'b010) begin //sw
      masking = 4'b1111;
      data_o = data_i;
   end
end

if (Load | data_valid)begin
   if(fun3==3'b000)begin //lb
      case (byteadd)
         00: begin
            wrap_load_out = {{24{wrap_load_in[7]}},wrap_load_in[7:0]};
         end
         01: begin
```

```
            wrap_load_out = {{24{wrap_load_in[15]}},wrap_load_in[15:8]};
        end
        10: begin
            wrap_load_out = {{24{wrap_load_in[23]}},wrap_load_in[23:16]};
        end
        11: begin
            wrap_load_out = {{24{wrap_load_in[31]}},wrap_load_in[31:24]};
        end
    endcase
end

if(fun3==3'b001)begin //lh
    case (byteadd)
        00: begin
            wrap_load_out = {{16{wrap_load_in[15]}},wrap_load_in[15:0]};
        end
        01: begin
            wrap_load_out = {{16{wrap_load_in[23]}},wrap_load_in[23:8]};
        end
        10: begin
            wrap_load_out = {{16{wrap_load_in[31]}},wrap_load_in[31:16]};
        end
    endcase
end

if(fun3==3'b010) begin //lw
    wrap_load_out = wrap_load_in;
end

if(fun3==3'b100)begin //lbu
    case (byteadd)
        00: begin
            wrap_load_out = {24'b0,wrap_load_in[7:0]};
        end
        01: begin
            wrap_load_out = {24'b0,wrap_load_in[15:8]};
        end
        10: begin
            wrap_load_out = {24'b0,wrap_load_in[23:16]};
        end
```

```
            11: begin
                wrap_load_out = {24'b0,wrap_load_in[31:24]};
            end
        endcase
    end

    if(fun3==3'b101)begin //lhu
        case (byteadd)
            00: begin
                wrap_load_out = {16'b0,wrap_load_in[15:0]};
            end
            01: begin
                wrap_load_out = {16'b0,wrap_load_in[23:8]};
            end
            10: begin
                wrap_load_out = {16'b0,wrap_load_in[31:16]};
            end
        endcase
    end

    if(fun3==3'b110) begin //lwu
        wrap_load_out = wrap_load_in;
    end
  end
end
endmodule
```

// STAGE 6 : Write back, common modules and Main processor
%%%

%%%

```
    output reg [31:0] out;

    always @ (*) begin
        case (sel)
            2'b00 : out = a;
            2'b01 : out = b;
            2'b10 : out = c;
            2'b11 : out = d;
        endcase
    end
endmodule


// mux3_8
module mux3_8 (a,b,c,d,e,f,g,h,sel,out);
    input wire [31:0] a,b,c,d,e,f,g,h;
    input wire [2:0] sel;

    output reg [31:0] out;

    always @ (*) begin
        case (sel)
            3'b000 : out = a;
            3'b001 : out = b;
            3'b010 : out = c;
            3'b011 : out = d;
            3'b100 : out = e;
            3'b101 : out = f;
            3'b110 : out = g;
            3'b111 : out = h;
        endcase
    end
endmodule
```

// STAGE 7 : Core and Microprocessor
%%%

```
wire jalr_decode;
wire next_sel_decode , next_sel_execute;
wire reg_write_decode , reg_write_execute , reg_write_memstage;
wire branch_result_decode , branch_result_execute;
wire [3:0]  mask;
wire [4:0]  alu_control_decode , alu_control_execute;
wire [1:0]  mem_to_reg_decode , mem_to_reg_execute;
wire [1:0]  mem_to_reg_memstage , mem_to_reg_wb;
wire [4:0]  rs1_decode , rs1_execute;
wire [4:0]  rs2_decode , rs2_execute;
wire [4:0]  rd_memstage , rd_wb;
wire [31:0] op_b_decode , op_b_execute , op_b_memstage;
wire [31:0] opa_mux_out_decode , opa_mux_out_execute;
wire [31:0] opb_mux_out_decode , opb_mux_out_execute;
wire [31:0] alu_res_out_execute , alu_res_out_memstage;
wire [31:0] alu_res_out_wb;
wire [31:0] next_sel_address_execute , next_sel_address_memstage;
wire [31:0] next_sel_address_wb;
wire [31:0] wrap_load_memstage , wrap_load_wb;
wire [31:0] rd_wb_data;
wire [31:0] alu_in_a , alu_in_b;

//FETCH STAGE
fetch u_fetchstage(
    .clk(clk),
    .rst(rst),
    .load(load_decode),
    .jalr(jalr_execute),
    .next_sel(next_sel_execute),
    .branch_reselt(branch_result_execute),
    .next_address(alu_res_out_execute),
    .instruction_fetch(instruction),
    .instruction(instruction_fetch),
    .address_in(0),
    .valid(data_mem_valid),
    .mask(instruc_mask_singal),
    .we_re(instruction_mem_we_re),
    .request(instruction_mem_request),
    .pre_address_pc(pre_pc_addr_fetch),
    .address_out(pc_address)
```

```
);

//FETCH STAGE PIPELINE
fetch_pipe u_fetchpipeline(
    .clk(clk),
    .rst(rst),
    .pre_address_pc(pre_pc_addr_fetch),
    .instruction_fetch(instruction_fetch),
    .pre_address_out(pre_pc_addr_decode),
    .instruction(instruction_decode),
    .next_select(next_sel_decode),
    .branch_result(branch_result_decode),
    .jalr(jalr_decode),
    .load(load_decode)
);

//DECODE STAGE
decode u_decodestage(
    .clk(clk),
    .rst(rst),
    .valid(data_mem_valid),
    .load_control_signal(load_execute),
    .reg_write_en_in(reg_write_wb),
    .instruction(instruction_decode),
    .pc_address(pre_pc_addr_decode),
    .rd_wb_data(rd_wb_data),
    .rs1(rs1_decode),
    .rs2(rs2_decode),
    .load(load_decode),
    .store(store_decode),
    .jalr(jalr_decode),
    .next_sel(next_sel_decode),
    .reg_write_en_out(reg_write_decode),
    .mem_to_reg(mem_to_reg_decode),
    .branch_result(branch_result_decode),
    .opb_data(op_b_decode),
    .instruction_rd(instruction_wb),
    .alu_control(alu_control_decode),
    .opa_mux_out(opa_mux_out_decode),
    .opb_mux_out(opb_mux_out_decode)
```

```
);

//DECODE STAGE PIPELINE
decode_pipe u_decodepipeline(
    .clk(clk),
    .rst(rst),
    .load_in(load_decode),
    .store_in(store_decode),
    .jalr_in(jalr_decode),
    .next_sel_in(next_sel_decode),
    .mem_to_reg_in(mem_to_reg_decode),
    .branch_result_in(branch_result_decode),
    .opb_data_in(op_b_decode),
    .alu_control_in(alu_control_decode),
    .opa_mux_in(opa_mux_out_decode),
    .opb_mux_in(opb_mux_out_decode),
    .pre_address_in(pre_pc_addr_decode),
    .instruction_in(instruction_decode),
    .reg_write_in(reg_write_decode),
    .rs1_in(rs1_decode),
    .rs2_in(rs2_decode),
    .reg_write_out(reg_write_execute),
    .jalr_out(jalr_execute),
    .load(load_execute),
    .store(store_execute),
    .next_sel(next_sel_execute),
    .mem_to_reg(mem_to_reg_execute),
    .branch_result(branch_result_execute),
    .opb_data_out(op_b_execute),
    .alu_control(alu_control_execute),
    .opa_mux_out(opa_mux_out_execute),
    .opb_mux_out(opb_mux_out_execute),
    .pre_address_out(pre_pc_addr_execute),
    .instruction_out(instruction_execute),
    .rs1_out(rs1_execute),
    .rs2_out(rs2_execute)
);

assign alu_in_a = (rs1_execute == rd_memstage )? alu_res_out_memstage :
                  (rs1_execute == rd_wb       )? alu_res_out_wb : opa_mux_out_execute;
```

```
assign alu_in_b = (rs2_execute == rd_memstage )? alu_res_out_memstage :
                  (rs2_execute == rd_wb       )? alu_res_out_wb : opb_mux_out_execute;

//EXECUTE STAGE
execute u_executestage(
    .a_i(alu_in_a),
    .b_i(alu_in_b),
    .pc_address(pre_pc_addr_execute),
    .alu_control(alu_control_execute),
    .alu_res_out(alu_res_out_execute),
    .next_sel_address(next_sel_address_execute)
);

//EXECUTE STAGE PIPELINE
execute_pipe u_executepipeline(
    .clk(clk),
    .rst(rst),
    .load_in(load_execute),
    .store_in(store_execute),
    .opb_datain(op_b_execute),
    .alu_res(alu_res_out_execute),
    .mem_reg_in(mem_to_reg_execute),
    .next_sel_addr(next_sel_address_execute),
    .pre_address_in(pre_pc_addr_execute),
    .instruction_in(instruction_execute),
    .reg_write_in(reg_write_execute),
    .reg_write_out(reg_write_memstage),
    .load_out(load_memstage),
    .store_out(store_memstage),
    .opb_dataout(op_b_memstage),
    .alu_res_out(alu_res_out_memstage),
    .mem_reg_out(mem_to_reg_memstage),
    .next_sel_address(next_sel_address_memstage),
    .pre_address_out(pre_pc_addr_memstage),
    .instruction_out(instruction_memstage)
);

//MEMORY STAGE
memory_stage u_memorystage(
    .rst(rst),
```

```
    .load(load_memstage),
    .store(store_memstage),
    .op_b(op_b_memstage),
    .instruction(instruction_memstage),
    .alu_out_address(alu_res_out_memstage),
    .wrap_load_in(load_data_in),
    .mask(mask),
    .data_valid(data_mem_valid),
    .valid(instruc_mem_valid),
    .we_re(data_mem_we_re),
    .request(data_mem_request),
    .store_data_out(store_data_out),
    .wrap_load_out(wrap_load_memstage)
);

assign rd_memstage = instruction_memstage[11:7];
assign rd_wb = instruction_wb[11:7];
assign alu_out_address = alu_res_out_memstage;
assign mask_singal = mask ;
assign load_signal = load_memstage;

//MEMORY STAGE PIPELINE
memory_pipe u_memstagepipeline(
    .clk(clk),
    .mem_reg_in(mem_to_reg_memstage),
    .wrap_load_in(wrap_load_memstage),
    .alu_res(alu_res_out_memstage),
    .next_sel_addr(next_sel_address_memstage),
    .pre_address_in(pre_pc_addr_memstage),
    .instruction_in(instruction_memstage),
    .reg_write_in(reg_write_memstage),
    .reg_write_out(reg_write_wb),
    .alu_res_out(alu_res_out_wb),
    .mem_reg_out(mem_to_reg_wb),
    .next_sel_address(next_sel_address_wb),
    .instruction_out(instruction_wb),
    .pre_address_out(pre_pc_addr_wb),
    .wrap_load_out(wrap_load_wb)
);
```

```
    //WRITE BACK STAGE
    write_back u_wbstage(
        .mem_to_reg(mem_to_reg_wb),
        .alu_out(alu_res_out_wb),
        .data_mem_out(wrap_load_wb),
        .next_sel_address(next_sel_address_wb),
        .rd_sel_mux_out(rd_wb_data)
    );
endmodule


// microprocessor
module microprocessor (
    input wire clk,
    input wire rst,
    input wire [31:0]instruction
    );

    wire [31:0] instruction_data;
    wire [31:0] pc_address;
    wire [31:0] load_data_out;
    wire [31:0] alu_out_address;
    wire [31:0] store_data;
    wire [3:0]  mask;
    wire [3:0]  instruc_mask_singal;
    wire instruction_mem_we_re;
    wire instruction_mem_request;
    wire instruc_mem_valid;
    wire data_mem_valid;
    wire data_mem_we_re;
    wire data_mem_request;
    wire load_signal;
    wire store;

    // INSTRUCTION MEMORY
    instruc_mem_top #(
        .INIT_MEM(1)
    )u_instruction_memory(
        .clk(clk),
        .rst(rst),
```

```
    .we_re(instruction_mem_we_re),
    .request(instruction_mem_request),
    .mask(instruc_mask_singal),
    .address(pc_address[9:2]),
    .data_in(instruction),
    .valid(instruc_mem_valid),
    .data_out(instruction_data)
);

//CORE
core u_core(
    .clk(clk),
    .rst(rst),
    .instruction(instruction_data),
    .load_data_in(load_data_out),
    .mask_singal(mask),
    .load_signal(load_signal),
    .instruc_mask_singal(instruc_mask_singal),
    .instruction_mem_we_re(instruction_mem_we_re),
    .instruction_mem_request(instruction_mem_request),
    .data_mem_we_re(data_mem_we_re),
    .data_mem_request(data_mem_request),
    .instruc_mem_valid(instruc_mem_valid),
    .data_mem_valid(data_mem_valid),
    .store_data_out(store_data),
    .pc_address(pc_address),
    .alu_out_address(alu_out_address)
);

// DATA MEMORY
data_mem_top u_data_memory(
    .clk(clk),
    .rst(rst),
    .we_re(data_mem_we_re),
    .request(data_mem_request),
    .address(alu_out_address[9:2]),
    .data_in(store_data),
    .mask(mask),
    .load(load_signal),
    .valid(data_mem_valid),
```

```
        .data_out(load_data_out)
    );
endmodule
```